

Warm-up: an email as a type, then five small types

Programming Paradigms
Session 3 — the warm-up, before the quest

Part 1, on paper, in pairs: about 15 minutes. Part 2, at the keyboard with `warmup.ml`: about 20 minutes; the rest at home.

The lecture said that an algebraic data type is a closed list of shapes, written where the compiler can read it, and that the compiler pays us back by checking every `match`. Before the music language, we try that on something everyone owns: a notification that goes out by email, by SMS or by push. Nothing here is themed, and in Part 1 there is nothing to run. The notation is OCaml's, and these are all we need:

<code>type t = A B C</code>	a <i>sum</i> : a value of <code>t</code> is A, or B, or C
<code>type t = A of int B of string</code>	a <i>sum</i> whose shapes carry something
<code>type t = { f : int; g : bool }</code>	a <i>product</i> : one value holds an <code>f</code> and a <code>g</code>
<code>int * bool</code>	a product without names, a pair
<code>t list, t option</code>	zero or more <code>t</code> ; a <code>t</code> or nothing (<code>Some x / None</code>)

Part 1 — on paper

Exercise 1 (the message)

Write OCaml types for the following, choosing *sum* or *product* as the facts demand. An **address** is a user name and a domain, both text. A **channel** is one of three: *email*, to an address; *SMS*, to a phone number; *push*, to a device token (both text). A **message** has a channel, a body, and a subject — which an SMS does not have. Then draw one concrete message sent by email as a tree: constructors at the nodes, text at the leaves. (If you write `type phone = string`, note that this synonym invents no values: every string is a phone. A *sum* does invent values — *Email*, *Sms*, *Push* did not exist before.)

Exercise 2 (why “algebraic”)

Let `type priority = Low | Normal | High`. How many distinct values are there of each type: `bool * bool`; `priority`; `priority * bool`; `priority option`; `bool option option`; `(priority * bool) option`? State the two rules that give the answers.

Exercise 3 (missing and unreachable)

Two functions on `channel`, written in a hurry:

```
let destination c = match c with
| Email a -> a.user ^ "@" ^ a.domain
| Sms number -> number

let is_push c = match c with
| Email _ | Sms _ | Push _ -> false
| Push _ -> true
```

For each: does it compile? What does the compiler print, and what is the message called? What does `destination (Push t)` do at run time, and what does `is_push (Push t)` return? Fix both.

Exercise 4 (the grid)

Three channels, *Email*, *Sms*, *Push*, and two things to do with each: *send* and *format*. Six cells of code. Draw the table, channels as rows and operations as columns. In OCaml, which cells sit together in one function? In Java, with one class per channel, which sit together in one class? Now the boss asks for two changes, separately: add WhatsApp as a channel, and add *schedule* as an operation. For each change, in each language: how many new cells, how many existing functions or classes are touched — and what does the compiler say if a cell is forgotten?

Part 2 — at the keyboard: warmup.ml

The file `warmup/warmup.ml` holds five small types with holes marked `TODO`, and a check for each; the exercises are stated here, the file only says where to type. It needs no `dune`:

```
ocaml warmup/warmup.ml          # pass / FAIL / todo, one line per check
```

Each rung is a few lines, and each is one match on one type.

Exercise 5 (a variant)

type weekday = Mon | Tue | Wed | Thu | Fri | Sat | Sun. Write next : weekday -> weekday (Sunday wraps to Monday) with one arm per day and no wildcard, and is_weekend : weekday -> bool with an or-pattern. Why prefer seven arms over | _ ->?

Exercise 6 (a record)

type point = { x : float; y : float }. Write dist : point -> point -> float and move_up : point -> float -> point, the latter with { p with y = ... }. After move_up p 1., what is p.y?

Exercise 7 (payloads, and a constructor added)

*type shape = Circle of float | Rect of float * float. Write area and perimeter. When both checks pass, add | Square of float to the type and run the file again without touching the functions. Copy down what the compiler prints. Then fix the functions, guided by what it printed.*

Exercise 8 (options)

Write safe_div : int -> int -> int option (None on division by zero) and max_of : int list -> int option (None on the empty list). In max_of, the recursive call returns an option: match on it.

Exercise 9 (a tree)

*type expr = Num of int | Add of expr * expr | Mul of expr * expr. Write eval : expr -> int and to_string : expr -> string so that Add (Num 1, Mul (Num 2, Num 3)) prints 1 + 2 * 3 and Mul (Add (Num 1, Num 2), Num 3) prints (1 + 2) * 3. When does a sum need parentheses? This tree — leaves and binary nodes — is the shape of the lab's melody, and the parentheses question returns in Part C2.*